goxpyriment — Timing Tests

christophe@pallier.org

2026-07-28

Contents

Timing-Tests: A Guide for Researchers

This document explains how to use the `Timing-Tests` program to characterise the timing behaviour of your computer before running a psychophysical experiment.

> **Quick reference** — flags, equipment, and one-line test descriptions are in `tests/Timing-Tests/README.md`. This document explains the *why*.

Why timing matters

In a psychology experiment every stimulus has an intended onset and offset time. Whether you are presenting a word for exactly 100 ms, playing a tone that should coincide with a visual flash, or measuring a reaction time to the nearest millisecond, you are trusting the computer to do two things correctly:

1. **Present stimuli when you ask it to.** If you ask for a 100 ms word, does the word actually appear on screen for 100 ms?

2. **Record timestamps accurately.** If you record the time of a key press, is that the time the key was physically depressed, or is it delayed by polling?

Neither is guaranteed. Both depend on your operating system, graphics driver, audio driver, and hardware. This suite lets you measure them on *your specific machine*, so you can report them in your methods section and fix problems before data collection begins.

Read this before you trust any number

The statistics these tests print come from software timestamps. They record when goxpyriment *believes* a flip happened — not what reached the panel.

This is not a theoretical caveat. In July 2026 a presentation bug on a GNOME/Wayland machine left the display showing stale frames for *seconds at a time*, while the program's own numbers stayed textbook-perfect throughout: bright-phase duration 199.915 ± 0.16 ms against a 200 ms target, cycle period 500.05 ± 2.45 ms. Nothing in the console output hinted at a problem. It was visible only with a photodiode. (Cause and fix are documented in `apparatus/CLAUDE.md`; the regression test is `tests/test_clear_only_frames`.)

So:

- **Software statistics tell you about the software.** They are genuinely useful for detecting dropped frames, scheduler jitter, and audio-buffer effects.
- **A photodiode and a trigger box tell you about the experiment.** Only they can confirm that light actually changed when you said it would.

If you are validating a rig for publication, measure it with hardware. The `av` test is built for exactly that.

The six tests

Two groups: what runs on any machine, and what needs equipment.

```
No hardware needed
  check    Go/no-go: can this machine display and make noise at all?
  display  What is my true refresh rate, and how stable are frame intervals?
  latency  How long does SDL hold audio before it sounds?

Photodiode and/or trigger box
  av       THE stimulus timing test — visual + audio + TTL, every cycle
  vrr      Can I present arbitrary durations, not just multiples of a frame?
  rt       How precise is my reaction-time measurement?
```

Two tests that used to live here now have their own directories: `tests/test_gv_sync` (`.gv` playback synchronisation) and `tests/test_dlpio8` (DLP-IO8-G square-wave characterisation).

This document assumes you have built the program:

```
go build -o Timing-Tests ./tests/Timing-Tests
```

Recording system information (`-sysinfo`)

Before running anything, capture a snapshot of the machine. `-sysinfo` prints it and exits — no window, no SDL:

```
Timing-Tests -sysinfo
```

```
Machine:    product: Precision 5490  System: Dell Inc.  Type: laptop
System:     Host: mylab-pc  Kernel: 7.0.0-28-generic x86_64  Uptime: 3h 12m
            OS: Ubuntu 26.04  Shell: bash 5.2.37  Desktop: ubuntu:GNOME
CPU:       Model: Intel(R) Core(TM) Ultra 7 165H  Info: 16 cores / 22 threads
Memory:     RAM: total: 30.04 GiB  used: 6.21 GiB (20.7%)
Graphics:  Card: Intel Corporation Meteor Lake-P [Intel Arc Graphics]  Driver: i915
Audio:      Server: PipeWire  v: 1.4.7
```

Keep it alongside your results:

```
Timing-Tests -sysinfo > sysinfo-$(hostname)-$(date +%Y%m%d).txt
```

Reviewers routinely ask for CPU model, OS version, graphics driver, and audio server. Every data file also carries this in its `-info.txt` header, including the **display the window actually opened on** — read it from there rather than assuming, especially on a multi-monitor machine.

Understanding the display refresh cycle

Most monitors refresh at a fixed rate — typically 60 Hz (a new frame every 16.67 ms), 120 Hz, or 144 Hz. Your graphics driver synchronises presentation to this cycle (VSYNC). Two consequences:

- **Durations are quantised to multiples of the frame period.** At 60 Hz you can show a stimulus for 16.67, 33.33, or 50 ms — but not 25 ms. Plan accordingly, or see the `vrr` test.
- **Onset is not when you call the flip; it is the next VSYNC**, anywhere from 0 to 16.67 ms later. goxpyriment keeps that offset constant once the pipeline is warm, but the first frames should be excluded — `-warmup` does this.

And the screen does not change all at once

An LCD paints top to bottom. On the rig this suite was developed against, a square at the bottom of the screen lights **13.15 ms** after one at the top — close to a full frame period. Measured on a 1280×1024 @ 60.02 Hz panel, that is **15.96 µs per pixel row**.

This matters more than it sounds. A stimulus at screen centre appears ~6.6 ms after one at the top. If your photodiode sits in a corner but your stimulus is central, your measured onset is biased by that amount — a systematic error larger than most of the jitter people worry about.

The `av` test draws five squares (four corners plus centre) precisely so you can measure this on your own display. Put a photodiode on a top square and another on a bottom one; the difference is your panel's scan-out gradient. Two squares on the same row are only microseconds apart and serve as a sanity check. Each display needs its own figure.

No hardware needed

`check` — does anything work at all

```
Timing-Tests -test check
```

Flashes a white screen for a second, then plays two sounds. If you do not see the flash or hear both sounds, stop and fix that first. Common causes: the window opened on the wrong monitor (`-d`), the audio device is muted, or the default output is not your speakers.

Measures nothing. It exists so you do not waste a session discovering the hardware was not plugged in.

`display` — true refresh rate and frame stability

```
Timing-Tests -test display -duration-s 30
```

Flips a uniform screen for `-duration-s` and reports the distribution of frame-to-frame intervals, plus the refresh rate implied by their mean. Use that measured rate for `-hz` in later tests rather than assuming 60.

What to look for: a tight distribution centred on your nominal frame period. A long right tail means dropped frames — a compositor, a background process, or power management. A *bimodal* distribution usually means the display is not running at the rate it reports.

`latency` — audio pipeline delay

```
Timing-Tests -test latency
Timing-Tests -test latency -audio-frames 256 -drain-reps 20
```

Plays tones of known duration and spin-polls until the audio stream has drained, measuring how long SDL holds PCM beyond the nominal duration. That difference is your audio output latency.

No hardware needed — but note it measures the *software* pipeline, not speaker-to-microphone delay. For true acoustic onset, use `av` with a microphone.

Photodiode and/or trigger box

`av` — the stimulus timing test

This is the one that matters. Every cycle presents all three modalities at once:

- **Visual** — five squares (four corners + centre) bright for `-frames-on` frames, then dark for `-frames-off` frames
- **Audio** — a tone lasting `frames-on × frame-period`, synchronised to the visual onset or offset from it by `-soa-ms`
- **TTL** — a `-trigger-ms` pulse at the visual onset

```
# 200 ms on, 300 ms off at 60 Hz, 100 cycles — the defaults
Timing-Tests -test av -cycles 100

# Visual only, no hardware at all
Timing-Tests -test av -no-sound -no-ttl

# Audio leading the flash by 50 ms
Timing-Tests -test av -soa-ms -50
```

`-no-sound` and `-no-ttl` drop a modality. This is how one test covers what used to be three: `-no-sound` is a pure visual-onset test, and the audio channel of a recording gives tone-onset jitter over a long session.

What to record. A photodiode on one square, the TTL line on a second channel, a microphone if you are checking audio. The quantity of interest is the *offset* between the TTL edge and the luminance step, and how stable it is across cycles. A constant offset is a calibration you subtract; a varying one is jitter you cannot.

What the console tells you. Bright-phase duration, cycle period, and (with sound) audio-queued minus visual onset. All software-side — cross-checks, not ground truth. See the warning at the top of this document.

Reference numbers. Dell Precision 5490, external 1280×1024 @ 60.02 Hz display under Wayland: 100/100 cycles clean, TTL→photodiode 29.878 ± 0.628 ms (from cycle 10 on), top-to-bottom gradient 13.150 ± 0.221 ms, ~5 dropped frames per 100 cycles. Same machine on a bare console with `SDL_VIDEODRIVER=kmsdrm`: 58/58 cycles, TTL→photodiode 21.51 ± 1.19 ms, **zero** dropped frames. Compositing costs roughly one extra frame of latency and a few percent of dropped frames.

Expect a warm-up transient. The first six cycles on that rig ran 36–39 ms before settling to ~27 ms. `-warmup 10` excludes them from the test's own statistics, but offline analysis tools do not know about it — quote steady-state numbers, and say which cycles you used.

Audio onsets are quantised by the buffer. With `-audio-frames 256` at 44100 Hz,

tone onsets land on 5.8 ms steps. This appears as a lattice in the inter-onset intervals and is not jitter — it is the buffer. Halving the buffer halves the step, at the cost of underrun risk.

`vrr` — arbitrary stimulus durations

```
Timing-Tests -test vrr -vrr-max-ms 50 -cycles 5
```

Why fixed refresh is a problem On a 60 Hz monitor every duration is a multiple of 16.67 ms. You can show a word for 16.67, 33.33, or 50 ms, but not 20 or 25 ms. At 120 Hz the quantum shrinks to 8.33 ms — better, but still coarse for subliminal work where you may want 10, 12, or 15 ms.

Variable Refresh Rate — AMD FreeSync, NVIDIA G-Sync, or the underlying VESA Adaptive-Sync standard — removes the quantisation. The display holds the current frame for exactly as long as you ask, then refreshes when told. Durations become controlled by your software timer rather than the display clock.

How the test works goxpyriment normally presents with VSync enabled, blocking until the next fixed edge. The `vrr` test switches the renderer to vsync=0 for the duration of the test (restored on exit), then sweeps target durations from 1 ms to `-vrr-max-ms` in 1 ms steps, holding each with a busy-wait and recording what was achieved.

Reading the result

- **On a working VRR display:** duration error stays small and roughly constant across the whole sweep.
- **On a fixed-refresh display:** the error is *periodic* — near zero at multiples of the frame period, rising in between, because the panel can only change on its own schedule. A sawtooth in the per-step output means you do not have VRR, whatever the monitor's box claimed.

The test prints one line per target plus an aggregate over the whole sweep.

```
# Does the connector support it? Look for "vrr_capable: 1"
sudo cat /sys/kernel/debug/dri/0/*/vrr_capable
```

Enabling VRR on Linux Confirm with the test itself rather than trusting the setting — that is the point of having it.

`rt` — reaction-time timestamp precision

```
Timing-Tests -test rt -cycles 50
```

Flashes the screen at a jittered interval and waits for a key press, recording the SDL3 hardware event timestamp against the flip timestamp. This characterises the *input* path: how much delay and jitter your keyboard, USB stack, and event queue add.

Press as fast as you can, or better, use a solenoid — you are characterising the machine, and human variability swamps it. The useful number is the SD, not the mean.

Interpreting results: what is "good"?

Metric	Excellent	Acceptable	Problematic
Frame-interval SD (`display`)	< 0.1 ms	< 0.5 ms	> 1 ms
Frames > 0.5 ms late (`display`)	< 1 %	< 5 %	> 10 %
Audio pipeline latency (`latency`)	< 15 ms	< 30 ms	> 50 ms
Bright-phase duration SD (`av`)	< 0.2 ms	< 1 ms	> 2 ms
TTL→photodiode SD (`av`, hardware)	< 0.5 ms	< 2 ms	> 5 ms
Dropped frames per 100 cycles (`av`)	0	< 5	> 10
VRR duration error SD (`vrr`)	< 0.1 ms	< 0.5 ms	> 1 ms (or periodic → no VRR)
RT SD (`rt`)	< 3 ms	< 10 ms	> 20 ms

The hardware rows are the ones worth quoting in a methods section.

Loading data in Python

Each run writes two files to `~/goxpy_data/`:

- `Timing-Tests_sub-000_date-<YYYYMMDD>-<HHMMSS>.csv` — data rows, no comments
- `...-info.txt` — session metadata: start/end time, hostname, OS, and the display and audio configuration actually in use

```
import pandas as pd

df = pd.read_csv("~/goxpy_data/Timing-Tests_sub-000_date-20260728-091541.csv")

# av: drop the warm-up cycles before quoting anything
steady = df[df.cycle >= 10]
print(steady.bright_duration_ms.describe())
print(steady.period_ms.std())
```

The `av` test writes one row per cycle: `cycle`, `t_visual_before_ms`, `t_visual_after_ms`, `bright_duration_ms`, `period_ms`, `t_audio_queued_ms`, `soa_intended_ms`, `soa_actual_ms`.

Always read display parameters from the run's own `-info.txt`. On a multi-monitor machine the displays differ in resolution, refresh rate and pixel density, and the scan-out calibration is per-display. The header records which one the window actually opened on.

Improving timing on your system

Linux

- **Disable the compositor.** By far the largest single improvement. Start from a virtual terminal (`Ctrl+Alt+F2`) with the window system stopped (`systemctl stop gdm`) and `SDL_VIDEODRIVER=kmsdrm`. On the reference rig this took dropped frames from ~5 % to zero and removed a frame of latency. Failing that, use a plain window manager (i3, openbox).
- Disable CPU frequency scaling: `cpupower frequency-set -g performance`.
- Run with real-time scheduling:

  ```
  sudo chrt -r 80 Timing-Tests -test display -duration-s 30
  ```

  For this you must first have edited `/etc/security/limits.conf`:

  ```
  # Change username to your login
  username    -       nice -20
  username    -       rtprio    99
  username    -       memlock   unlimited
  ```

  Changes apply after login; check with `ulimit -r`, which should return `99`. Use `@goxpy` instead of a username to grant this to a group (`usermod -aG goxpy $USER`).
- Consider a real-time kernel — see https://ubuntu.com/blog/enable-real-time-ubuntu.
- Reduce USB trigger latency — see the DLP-IO8 section below.

Windows

- Disable "Hardware-Accelerated GPU Scheduling" in Display Settings if you see high frame jitter.

Audio buffer size

The hardware audio buffer is controlled by the SDL hint `SDL_AUDIO_DEVICE_SAMPLE_FRAMES`, exposed as `-audio-frames`. It must be set before the audio device opens.

```
# Default (platform-dependent, often 512–2048 frames)
Timing-Tests -test latency

# Aggressive low-latency (~5.8 ms at 44100 Hz)
```

```
Timing-Tests -test latency -audio-frames 256 -drain-reps 20

# Conservative (~46 ms, stable anywhere)
Timing-Tests -test latency -audio-frames 2048
```

On startup the program prints the actual device format:

```
audio: 44100 Hz  2 ch  256 sample frames (~5.8 ms latency)
```

The buffer sets the floor on audio-onset precision: onsets can only land on buffer boundaries, so 256 frames at 44100 Hz quantises them to 5.8 ms steps. Use `latency` at several sizes to find the smallest that drains stably (low SD), then use that everywhere — including in your actual experiment.

Related tests

- `tests/test_gv_sync` — `.gv` video playback: does `PlayGvFunc` put a frame on screen when it says it does?
- `tests/test_dlpio8` — square-wave output for characterising the trigger box itself against an oscilloscope.
- `tests/test_clear_only_frames` — regression test for the compositor presentation bug described at the top of this document.
- `tests/test_vsync_blocking` — does `SDL_RenderPresent` block on VSYNC on this platform, or return immediately?

DLP-IO8

See https://github.com/chrplr/dlp-io8-g.

Parallel port alternative

If you have a parallel port (LPT) at `/dev/parport0`, use `triggers.NewParallelPort("/dev/parport0"` in your experiment code. The `Send(byte)` method sets all 8 data lines simultaneously.
On Linux you need `sudo modprobe ppdev` and membership in the `lp` group.